

Experiment: 01

1. Objectives:

1. To understand and implement Newton's Forward Difference Interpolation in Python.
2. To understand and implement Newton's Backward Difference Interpolation in Python.
3. To compare the results and accuracy of both techniques.

2. Introduction:

Newton's Forward Difference Interpolation is a numerical method used to estimate the value of a function at an intermediate point using equally spaced data points. It constructs a polynomial by starting from the first data point and using forward differences (Δ , Δ^2 , Δ^3 , ...).

$$y(x) = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!} \cdot \Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!} \cdot \Delta^3 y_0 + \frac{p(p-1)(p-2)(p-3)}{4!} \cdot \Delta^4 y_0 + \dots$$

Newton's Backward Difference Interpolation is a numerical technique that estimates function values at non-tabulated points using equally spaced data. It builds the interpolating polynomial starting from the last data point and employs backward differences (∇ , ∇^2 , ∇^3 , ...).

$$y(x) = y_n + p\nabla y_n + \frac{p(p+1)}{2!} \cdot \nabla^2 y_n + \frac{p(p+1)(p+2)}{3!} \cdot \nabla^3 y_n + \frac{p(p+1)(p+2)(p+3)}{4!} \cdot \nabla^4 y_n + \dots$$

3. Procedure

Algorithm:

Experiment 1: Newton's Forward Difference Interpolation

1. Start
2. Define the function
3. Compute step size
4. Build forward difference table
5. Compute $p = (x_i - x[0]) / h$
6. Apply Newton forward formula
7. Display interpolated value and difference tables
8. End

Experiment 2: Newton's Backward Difference Interpolation

1. Start
2. Define the function
3. Compute step size
4. Build backward difference table
5. Compute $p = (x_i - x_{[n-1]}) / h$
6. Apply Newton backward formula
7. Display interpolated value and difference tables
8. End

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** PyCharm Community Edition 20251.1.1

5. Experiment Implementation:

Code 1: Newton's Forward Difference Interpolation

```
def forward_diff_table(y):
    n = len(y)
    table = [[0 for _ in range(n)] for _ in range(n)]

    # First column is f(x)
    for i in range(n):
        table[i][0] = y[i]

    # Compute forward differences
    for j in range(1, n):
        for i in range(n - j):
            table[i][j] = table[i + 1][j - 1] - table[i][j - 1]

    return table

def newton_forward_interpolation(x, y, xi):
    n = len(x)
    h = x[1] - x[0] # equal spacing assumed
    p = (xi - x[0]) / h

    table = forward_diff_table(y)
    result = y[0]
    p_term = 1

    for i in range(1, n):
        p_term *= (p - (i - 1))
        result += (p_term * table[0][i]) / factorial(i)

    return result, table
```

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    f = 1
    for i in range(2, n + 1):
        f *= i
    return f

x = [1891, 1901, 1911, 1921, 1931]
y = [46,66,81,93,101]
xi = 1895

interpolated_value, table = newton_forward_interpolation(x, y,
xi)
print(f"\nNewton Forward Interpolation at x = {xi:.4f} :
{interpolated_value:.4f}")
```

Output:

C:\Users\Rezwan\PyCharmMiscProject\.venv\Scripts\python.exe

Newton Forward Interpolation at x = 1895.0000 : 54.8528

Process finished with exit code 0

Code 2: Newton's Backward Difference Interpolation

```
def backward_diff_table(y):
    n = len(y)
    table = [[0 for _ in range(n)] for _ in range(n)]

    # First column is f(x)
    for i in range(n):
        table[i][0] = y[i]

    # Compute backward differences
    for j in range(1, n):
        for i in range(j, n):
            table[i][j] = table[i][j - 1] - table[i - 1][j - 1]

    return table

def newton_backward_interpolation(x, y, xi):
    n = len(x)
    h = x[1] - x[0] # equal spacing assumed
    p = (xi - x[-1]) / h # note xi - x_n

    table = backward_diff_table(y)
    result = y[-1]
    p_term = 1

    for i in range(1, n):
        p_term *= (p + (i - 1))
        result += (p_term * table[-1][i]) / factorial(i)

    return result, table
```

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    f = 1
    for i in range(2, n + 1):
        f *= i
    return f

x = [1891, 1901, 1911, 1921, 1931]
y = [46, 66, 81, 93, 101]
xi = 1895

interpolated_value, table = newton_forward_interpolation(x, y,
xi)
print(f"\nNewton Forward Interpolation at x = {xi:.4f} :
{interpolated_value:.4f}")
```

Output :

```
C:\Users\Rezwan\PyCharmMiscProject\.venv\Scripts\python.exe

Newton Backward Interpolation at x = 1895.0000 : 54.8528

Process finished with exit code 0
```

6. Discussion & Conclusion:

In this experiment, Newton's Forward and Backward Interpolation methods were implemented to estimate the population in 1895 using census data from 1891 to 1931. Both produced identical and accurate interpolated values. These methods are highly effective for equally spaced data and provide a systematic way to estimate function values between known points. They are foundational in numerical analysis and widely used in engineering and scientific computations.

7. Reference:

1. <https://math.stackexchange.com/questions/624894/newtons-interpolation-formula-difference-between-the-forward-and-the-backward>